PREZENTAREA 1 DIN 6 · CURS 1–2 · LABORATOR 0–3

Primii pași în Haskell

Sintaxă, tipuri, funcții și liste

ghci

```
ghci> sum [x * x | x <- [1..10], even x]
220
```

Ce facem azi

01

De ce Haskell

imperativ vs. declarativ, ideile limbajului

Curs 1

02

Mediul de lucru

GHCi, fișiere .hs, Hoogle

Lab 0–1

03

Sintaxă și legări

indentare, imutabilitate, let și where

Curs 1 · Lab 2

04

Tipuri

tipuri de bază, tupluri, clase, erori de tip

Curs 2

05

Funcții

semnături, pattern matching, gărzi, recursie

Curs 2 · Lab 2

06

Liste

(:), șabloane, descrieri de liste, lazy

Curs 2 · Lab 3

07

Exerciții

opt probleme din laboratoare, cu soluții

Lab 2 · Lab 3

08

Recapitulare

capcane, fițuică, quiz

01

De ce Haskell

Ce înseamnă programare funcțională și ce face Haskell diferit de limbajele pe care le știi deja.



Imperativ vs. declarativ

Imperativ — CUM

Îi dai calculatorului pașii, unul după altul.

Python

```python
total = 0
for x in xs:
    if x % 2 == 0:
        total += x * x
```

Stare care se schimbă la fiecare pas (total +=).

Declarativ — CE

Descrii rezultatul pe care îl vrei.

Haskell

```haskell
total = sum [x * x | x <- xs, even x]
```

Nicio variabilă nu se modifică. Citești: „suma pătratelor lui x, pentru x din xs, x par”.

Analogie din curs: **rețeta pas cu pas** vs. **„vreau paste”** la restaurant: tu spui ce vrei, bucătarul decide cum.

Haskell în patru idei



Funcții pure

Aceeași intrare dă mereu același rezultat. O funcție nu modifică nimic în jurul ei: nu citește fișiere, nu afișează nimic pe ascuns.



Funcțiile sunt valori

Le poți da ca argumente și le poți întoarce ca rezultat. Pe asta se construiesc map, filter și fold (prezentarea 2).



Evaluare lenevă (lazy)

O expresie se calculează doar când e nevoie de rezultat și doar cât e nevoie. Așa poți lucra cu liste infinite: [1..].



Tipare puternică și statică

Orice expresie are un tip. Compilatorul verifică tipurile înainte de rulare și le deduce singur.

Un prim exemplu: quicksort în șase linii

qsort.hs

```
qsort :: Ord a => [a] -> [a]
qsort []     = []
qsort (p:xs) = qsort lesser ++ [p] ++ qsort greater
  where
    lesser  = filter (< p) xs
    greater = filter (>= p) xs
```

ghci

```
ghci> qsort [3,1,4,1,5,9,2,6]
[1,1,2,3,4,5,6,9]
```

1. **Semnătura**
   orice tip a care are ordine (Ord) → listă sortată
2. **Cazul de bază**
   lista vidă e deja sortată
3. **Șablonul (p:xs)**
   separă primul element (pivotul) de rest
4. **where**
   definește lesser și greater local, după corp
5. **++**
   lipește: mici, pivot, mari

02

Mediul de lucru

Instalare cu GHCup, interpretorul GHCi, fișiere sursă .hs și căutare în Hoogle (Laboratoarele 0 și 1).



GHCup instalează tot ce îți trebuie



GHC

compilatorul Glasgow Haskell; vine cu GHCi, interpretorul interactiv



Cabal

gestionează proiecte și dependențe (biblioteci)



HLS

Haskell Language Server: erori, tipuri și completare în editor



Stack

proiecte cu versiuni diferite de GHC, plus teste

terminal

```
$ ghci
ghci> 2 + 3
5
ghci> False || True
True
ghci> head "abcd"
'a'
ghci> tail "abcd"
"bcd"
ghci> :q
```

Instalare: `haskell.org/ghcup` · Editor: VS Code + extensia **Haskell**

Comenzile GHCi pe care le folosești mereu

`:l fisier.hs`	încarcă (load) un fișier sursă
`:r`	reîncarcă (reload) după ce ai modificat fișierul
`:t expr`	afișează tipul unei expresii
`:i nume`	informații: definiție, clasă, instanțe
`:m + Data.List`	adaugă un modul (sau import Data.List)
`:q`	ieșire (quit) · :? afișează ajutorul
`Ctrl + C`	oprește o evaluare care durează prea mult

ghci

```
ghci> :t True
True :: Bool
ghci> :t not
not :: Bool -> Bool
ghci> :i Bool
data Bool = False | True
-- ... și instanțele: Eq, Ord, Show, ...
```

Fluxul de lucru cu fișiere .hs

1 **Scrii** codul în lab1.hs → **2** **`:l lab1.hs`** îl încarci în GHCi → **3** **Testezi** apelezi funcțiile → **4** **`:r`** după modificări

lab1.hs

```
myInt = 3141592653589793238462643383279502884197169399375105820974944592 3

double :: Integer -> Integer
double x = x + x
triple :: Integer -> Integer   -- o adaugi tu
triple x = 3 * x
```

ghci

```
ghci> double myInt
error: Variable not in scope: double
ghci> :l lab1.hs
Ok, 1 module loaded.
ghci> double myInt
628318530717958647692528676655900576839433879875021164194988918 46
```

Hoogle: caută după nume sau după tip

[a] -> [[a]]

permutations :: [a] -> [[a]]

Data.List

subsequences :: [a] -> [[a]]

Data.List

inits :: [a] -> [[a]]

Data.List

tails :: [a] -> [[a]]

Data.List

ghci

```
ghci> import Data.List
ghci> permutations [1,2,3]
[[1,2,3],[2,1,3],[3,2,1],[2,3,1],[3,1,2],[1,3,2]]
ghci> subsequences "abc"
["","a","b","ab","c","ac","bc","abc"]
```

De reținut

- Nu știi numele? Caută după **tip**.
- Funcțiile din `Data.List` cer `import Data.List` (în GHCi sau sus în fișier).

03

Sintaxă și legări

Comentarii, identificatori, indentare, variabile imutabile, let … in și where (Curs 1, Laborator 2).



Sintaxa de bază

Comentarii

```
-- pe o linie

{- pe mai
   multe linii -}
```

Ca // și /* */ din C.

Identificatori

```
double x = 2 * x
fact' n = n   -- ok
data Point = Pt Int Int
```

Literă mică: variabile și funcții.
Majusculă: tipuri și constructori.

Case sensitive

```
x = 1
X = 1   -- eroare!
-- X ar fi constructor
```

x și X sunt nume diferite, iar X nici nu poate fi variabilă.



Literă mică sau majusculă nu e o chestiune de stil, ci de sintaxă: prima literă îi spune compilatorului dacă numele e o valoare sau un tip/constructor.

Indentarea face parte din sintaxă

Trei scrieri echivalente

aliniere sau acolade și ;

```
trei = let a = 1
           b = 2
       in a + b

trei = let {a = 1; b = 2} in a + b

trei = let a = 1; b = 2 in a + b
```

a și b încep pe aceeași coloană: sunt în același bloc.

Greșeala din Laboratorul 2

Lab2.hs

```
double :: Integer -> Integer
   double x = x + x
```

ghci

```
ghci> :r
error: parse error on input ‘=’
```

De ce? Linia indentată e citită ca o continuare a semnăturii de deasupra. Definițiile de la nivelul fișierului încep în coloana 1.

Variabilele sunt imutabile

În Haskell, = înseamnă **legare** (binding), nu atribuire.

test.hs

```
x = 1
x = 2
```

ghci

```
ghci> :l test.hs
error: Multiple declarations of ‘x’
```



În GHCi, x = 2 după x = 1 „merge”: nu modifică x, ci creează o legare nouă care o ascunde pe prima.

C, Java, Python

x este o cutie; îi poți schimba conținutul oricând (x = x + 1).

Haskell

x este un nume pentru o valoare. Odată legat, nu se mai schimbă.

let … in vs. where

`let … in`

o expresie: poate apărea oriunde e voie o valoare; definițiile vin înainte

```
z = let x = 3 in x + 1      -- 4

lista = [let y = 8 in y, 9]
-- [8,9]
```

Expresiile `if … then … else` și `case … of` sunt și ele expresii, la fel ca `let`.

`where`

o clauză a unei definiții: vine după corp, vizibilă în toate gărzile

```
f x = g x + g x + z
  where
    g y = 2 * y
    z   = x - 1
-- f 1 == 4
```

```
lista = [y where y = 8, 9]
-- parse error: nu e expresie
```

Quiz: ce valoare are?

1 Ce valori au z și x?

```
x = 1
z = let x = 3 in x
```

2 Ce valoare are x?

```
x = let z = 5
        g u = z + u
    in let z = 7
       in g 0 + z
```

3 Dar acum?

```
x = let z = 5; g u = z + u
    in let z = 7 in g 0
```

Încearcă fără GHCi, doar cu creionul. Răspunsurile sunt pe slide-ul următor.

Quiz: răspunsuri

1

`z = 3,  x = 1`

let x = 3 creează un x nou, vizibil doar în expresia de după in. Variabila x de sus nu se schimbă.

2

`x = 12`

g a fost definită unde z = 5, așa că g 0 = 5. Apoi + z folosește z = 7: 5 + 7 = 12.

3

`x = 5`

Același g 0 = 5. Fără + z, rezultatul e 5: z = 7 nu ajunge în g.

Regula: o funcție vede variabilele din locul în care a fost **definită**, nu din locul în care e **apelată** (domeniu static).

04

Tipuri

Tipuri de bază, tupluri și liste, clase de tipuri, cum citești o eroare de tip (Curs 2).



Sistemul de tipuri: puternic, static, dedus

Puternic

Nicio conversie implicită: nu aduni un Bool cu un Char, nici un Int cu un Double fără conversie.

Static

Tipul fiecărei expresii e calculat la compilare. Greșelile apar la :l, nu în timpul rulării.

Dedus automat

Nu ești obligat să scrii tipurile; compilatorul le deduce. Scrie-le totuși pentru funcțiile de top.

ghci

```
ghci> :t ('a', True)
('a', True) :: (Char, Bool)
ghci> :t not
not :: Bool -> Bool
```

Citește `::` ca „are tipul”.

not :: Bool -> Bool = „not este o funcție de la Bool la Bool”.

Tipurile de bază

Tip	Exemple	De reținut
`Int`	`42, -7`	întreg pe 64 de biți: poate da overflow
`Integer`	`2 ^ 100`	întreg oricât de mare (myInt din Lab 1)
`Float, Double`	`3.14`	virgulă mobilă; Double are precizie dublă
`Bool`	`True, False`	data Bool = False | True
`Char`	`'a', '\n'`	un caracter, între apostrofuri
`String`	`"Tom"`	type String = [Char]: o listă de caractere

ghci

```
ghci> (maxBound :: Int) + 1
-9223372036854775808
ghci> 2 ^ 64 :: Integer
18446744073709551616
ghci> sqrt (fromIntegral (4 :: Int))
2.0
ghci> 'a' : "bc"
"abc"
```



Int vs Integer: Int sare la valori negative fără avertisment. Când numerele pot fi mari, folosește Integer.

Tupluri vs. liste

Tuplu `(a, b, c)`

- număr fix de componente
- componentele pot avea tipuri diferite
- tipul conține fiecare componentă

```
ghci> :t (1 :: Int, 'a', "ab")
(1 :: Int, 'a', "ab") :: (Int, Char, String)
ghci> fst (1, 'a')     -- doar perechi
1
ghci> snd (1, 'a')
'a'
```

Listă `[a]`

- oricâte elemente, chiar zero
- toate elementele au același tip
- tipul nu spune câte elemente are

```
ghci> :t [True, False, True]
[True, False, True] :: [Bool]
ghci> [1, 'a']
error: No instance for (Num Char)
ghci> length "Tom"
3
```

Cum citești un tip cu clasă

```
1 :: Num a => a
```

- `1` — valoarea
- `Num a` — constrângere: a e din clasa Num
- `a` — variabilă de tip: orice tip numeric

```
maxim :: Ord p => p -> p -> p
```

- `maxim` — numele
- `Ord p` — p are ordine (<, >)
- `p -> p -> p` — două argumente de tip p
- `p` — rezultat de tip p

Clase întâlnite azi

`Num`	numere (+, *)
`Ord`	ordine (<, >, max)
`Eq`	egalitate (==, /=)



Citește cu voce tare: „pentru orice tip `a` din clasa `Num`, 1 are tipul `a`". Așa poate fi 1 un Int, un Integer sau un Double, după context.

Erori de tip: citește-le, nu te speria

ghci · exemplul 1

```
ghci> "abc" ++ 'd'
error: Couldn't match expected type ‘[Char]’
         with actual type ‘Char’
ghci> "abc" ++ "d"
"abcd"
```

ghci · exemplul 2

```
ghci> xs = [1, 2, 3] :: [Int]
ghci> sqrt (length xs)
error: No instance for (Floating Int)
ghci> sqrt (fromIntegral (length xs))
1.7320508075688772
```

Cum o citești

1. **Unde?**
   linia și coloana din mesaj
2. **Ce aștepta?**
   expected type: tipul cerut acolo
3. **Ce a primit?**
   actual type: tipul pe care l-ai dat
4. **Verifică**
   :t pe fiecare bucată a expresiei

05

Funcții

Semnături, aplicare, pattern matching, gărzi, case și recursivitate (Curs 2, Laborator 2).



Anatomia unei funcții

două argumente

```
add :: Integer -> Integer -> Integer
add x y = x + y
```

aplicare

```
ghci> add 3 7
10
```

un argument de tip tuplu

```
dist :: (Integer, Integer) -> Integer
dist (a, b) = abs (a - b)        -- un argument: o pereche

ghci> dist (5, 7)                -- 2
```

Semnătura

nume :: tipuri

Definiția

parametri = corp

Aplicarea

fără paranteze, fără virgule

⚠ `add(3, 7)` nu e un apel cu două argumente: e add aplicat pe tuplul (3, 7) → eroare de tip.

Citește săgețile

```
foo :: a -> b -> [a] -> [b]
```

arg. 1 · arg. 2 · arg. 3 · rezultat

```
map :: (a -> b) -> [a] -> [b]
```

arg. 1: o funcție · arg. 2: o listă · rezultat

exersează

```
ghci> :t (:)
(:) :: a -> [a] -> [a]
ghci> :t (++)
(++) :: [a] -> [a] -> [a]
ghci> :t zip
zip :: [a] -> [b] -> [(a, b)]
```

Regula: ultimul tip din lanț e rezultatul; cele dinainte sunt argumentele. Parantezele grupează un argument care este el însuși o funcție.

Trei moduri de a scrie fact

if … then … else

```
fact :: Integer -> Integer
fact n =
  if n == 0
    then 1
    else n * fact (n - 1)
```

else e obligatoriu: if e o expresie, deci trebuie să aibă mereu o valoare.

Pattern matching

```
fact :: Integer -> Integer
fact 0 = 1
fact n = n * fact (n - 1)
```

Ecuațiile sunt încercate de sus în jos; prima care se potrivește câștigă.

Gărzi

```
fact :: Integer -> Integer
fact n
  | n == 0    = 1
  | otherwise =
      n * fact (n - 1)
```

Condițiile se testează în ordine. otherwise e doar un alt nume pentru True.

În practică: pattern matching pentru cazurile pe valori concrete, gărzi pentru condiții. `if` rămâne pentru alegeri simple, pe un rând.

Pattern matching: ordinea contează

Ordine greșită

fact.hs

```
fact :: Integer -> Integer
fact n = n * fact (n - 1)   -- prinde și 0!
fact 0 = 1                  -- nu se ajunge aici
```

n se potrivește cu orice valoare, inclusiv 0, deci recursia nu se mai oprește.

Regula: pune cazurile particulare (0, [], True) **înaintea** celor generale (n, xs, _).

Trei feluri de șabloane

din Curs 2

```
(||) :: Bool -> Bool -> Bool
False || x = x
True  || _ = True
```

`True, False, 0` — constructori și constante: se potrivesc doar cu ei înșiși

`x, n, xs` — variabile: se potrivesc cu orice și leagă valoarea

`_` — wildcard: se potrivește cu orice, fără să lege nimic

Gărzi, where și case

Gărzi + where

```haskell
h :: Integer -> Integer
h x
  | x == 0    = 0
  | x == 1    = y + 1
  | x == 2    = y * y
  | otherwise = y
  where y = x * x
```

Condiții booleene oarecare, testate de sus în jos.

case ... of + where

```haskell
f :: Integer -> Integer
f x = case x of
    0 -> 0
    1 -> y + 1
    2 -> y * y
    _ -> y
  where y = x * x
```

Pattern matching pe o expresie, în interiorul corpului. where e mai puțin indentat decât ramurile.

În ambele variante, `where` de la final e vizibil în toate ramurile.

Recursivitate în loc de bucle

cu gărzi (Lab 2)

```
fibonacciCazuri :: Integer -> Integer
fibonacciCazuri n
  | n < 2     = n
  | otherwise = fibonacciCazuri (n - 1)
              + fibonacciCazuri (n - 2)
```

ecuațional, cu șabloane (Lab 2)

```
fibonacciEcuational :: Integer -> Integer
fibonacciEcuational 0 = 0
fibonacciEcuational 1 = 1
fibonacciEcuational n =
  fibonacciEcuational (n - 1) + fibonacciEcuational (n - 2)
```

Rețeta

1. **Cazul de bază**
   când te oprești: 0, 1, lista vidă
2. **Cazul general**
   aceeași problemă, pe date mai mici
3. **Încredere**
   presupune că apelul recursiv e corect; combină rezultatele

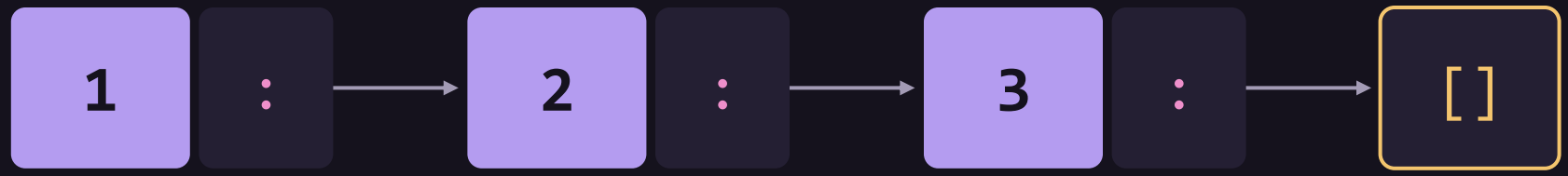
06

Liste

Constructorii [] și (:), operații, șabloane, descrieri de liste, zip și evaluare leneșă (Curs 2, Laborator 3).



O listă e construită din [] și (:)



cap (head)

coada (tail) = restul listei

aceeași listă, scrisă în mai multe feluri

```
[1,2,3] == 1 : (2 : (3 : []))
        == 1 : 2 : 3 : []

"abc"   == ['a','b','c']
        == 'a' : 'b' : 'c' : []
```

Definiția recursivă

- lista vidă: `[]`
- sau un element `x` (capul) pus în fața unei liste `xs` (coada): `x : xs`

Operații și intervale

```
head [1,2,3]
→ 1
```

```
tail [1,2,3]
→ [2,3]
```

```
[1,2,3] !! 2
→ 3
```

```
[1,2] ++ [3]
→ [1,2,3]
```

```
take 2 [5,6,7]
→ [5,6]
```

```
drop 2 [5,6,7]
→ [7]
```

```
length "abc"
→ 3
```

```
reverse [1,2,3]
→ [3,2,1]
```

```
3 `elem` [1,2,3]
→ True
```

```
null []
→ True
```

Intervale și progresii

```
ghci> [1..5]
[1,2,3,4,5]
ghci> ['c'..'e']
"cde"
ghci> [20,17..1]
[20,17,14,11,8,5,2]
ghci> [2.0,2.5..4.0]
[2.0,2.5,3.0,3.5,4.0]
```

Primul element, al doilea (dă pasul), limita.

Șabloane pentru liste

exemplu

```
lungime :: [a] -> Int
lungime []     = 0
lungime (_:xs) = 1 + lungime xs
```

`[]`	lista vidă
`(x:xs)`	nevidă: cap x, coadă xs
`[x]`	exact un element
`(x:y:rest)`	cel puțin două elemente

Capcane

```
f x:xs = ...      -- greșit
f (x:xs) = ...    -- corect
```

Fără paranteze, f x:xs se citește (f x) : xs.

```
foo x x = x       -- eroare
foo x y
  | x == y = x    -- corect
```

Șabloane liniare: Conflicting definitions for ‘x’.

Exemplu din Lab 3: semiPare în două stiluri

recursiv

```haskell
semiPareRec :: [Int] -> [Int]
semiPareRec [] = []
semiPareRec (h:t)
  | even h    = h `div` 2 : t'
  | otherwise = t'
  where t' = semiPareRec t
```

descriere de listă

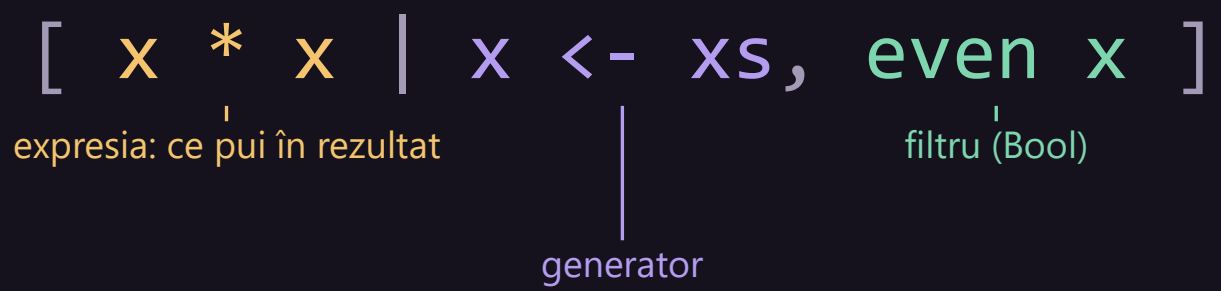
```haskell
semiPareComp :: [Int] -> [Int]
semiPareComp l = [x `div` 2 | x <- l, even x]
```

ghci

```
ghci> semiPareRec [0,2,1,7,8,56,17,18]
[0,1,4,28,9]
ghci> semiPareComp [0,2,1,7,8,56,17,18]
[0,1,4,28,9]
```

Cum se corespund: filtrul `even x` devine o gardă, iar transformarea `x `div` 2` se pune în fața rezultatului pentru coadă.

Descrieri de liste (list comprehensions)



din Curs 2

```
ghci> [x | x <- [0..10], even x]
[0,2,4,6,8,10]
ghci> [(x,y) | x <- [0..6], y <- [0..6],
               x + y == 10]
[(4,6),(5,5),(6,4)]
ghci> [(i,j) | i <- [1..2], let k = 2 * i,
               j <- [1..k]]
[(1,1),(1,2),(2,1),(2,2),(2,3),(2,4)]
```

Forma generală: `[ expresie | selectori, legări, filtrări ]`

- **selectori:** `pattern <- lista`, unul sau mai mulți
- **legări:** `let k = 2 * i`
- **filtrări:** expresii Bool; elementele cu False se aruncă

zip vs. doi generatori

zip: în paralel

```
ghci> zip [1..3] "ABCD"
[(1,'A'),(2,'B'),(3,'C')]
```

Perechi pe poziție; se oprește la lista cea mai scurtă.

Doi generatori: toate combinațiile

```
ghci> [(x,y) | x <- [1..3], y <- "AB"]
[(1,'A'),(1,'B'),(2,'A'),(2,'B'),
 (3,'A'),(3,'B')]
```

Produs cartezian: al doilea generator variază cel mai repede.

Trucul pentru poziții

```
ghci> [x | (i, x) <- zip [1..] ['A'..'Z'], even i]
"BDFHJLNPRTVXZ"
```

Evaluare lenesă și liste infinite

liste infinite

```
ghci> take 5 [0..]
[0,1,2,3,4]
ghci> take 7 [0,2..]
[0,2,4,6,8,10,12]
ghci> take 3 (zip [1,1..] [0,0..])
[(1,0),(1,0),(1,0)]
```

doar cât e nevoie

```
ghci> f a = 5
ghci> f (head [])
5
ghci> [1, head [], 3] !! 0
1
ghci> [1, head [], 3] !! 1
*** Exception: Prelude.head: empty list
```



Lazy: argumentele se evaluează doar **când** e nevoie și doar **cât** e nevoie. Atenție: `length [0..]` nu se termină niciodată (oprește cu Ctrl+C).

Operatorii sunt funcții cu două argumente

operator nou

```
(&&&) :: Bool -> Bool -> Bool
True  &&& b = b
False &&& _ = False
```

infix ↔ prefix

```
ghci> (+) 2 3            -- operator → prefix
5
ghci> 5 `mod` 2          -- funcție → infix
1
ghci> 1 `elem` [1,2,3]
True
```

Precedență (mare = leagă mai strâns)

9	`!!` `.`	
8	`^`	dreapta
7	`*` `/` `` `div` `` `` `mod` ``	stânga
6	`+` `-`	stânga
5	`:` `++`	dreapta
4	`==` `/=` `<` `<=` `` `elem` ``	
3	`&&`	dreapta
2	`||`	dreapta

```
5 - 2 - 1  == (5 - 2) - 1    -- stânga
1 : 2 : [] == 1 : (2 : [])   -- dreapta
```

07

Exerciții

Opt probleme din Laboratoarele 2 și 3. Încearcă singur întâi; soluțiile urmează.



Setul de exerciții

1 L2·9

fizzbuzz

„Fizz” pentru multiplii de 3, „Buzz” pentru 5, „FizzBuzz” pentru ambii, altfel "". Cu if și cu gărzi.

2 L2·10

tribonacci

T1 = T2 = 1, T3 = 2, apoi Tn = suma ultimilor trei termeni.

3 L2·11

binomial

B(n,k) = B(n-1,k) + B(n-1,k-1); B(n,0) = 1; B(0,k) = 0.

4 L3·3

nrVocale

Numărul de vocale din cuvintele palindrom. ["sos", "civic", "palton"] → 3

5 L3·4

după fiecare par

f 3 [1,2,3,4,5,6] → [1,2,3,3,4,3,5,6,3]

6 L3·5–6

listadiv

Lista divizorilor fiecărui număr. [1,4,6] → [[1],[1,2,4],[1,2,3,6]]

7 L3·9

pozitiiImpare

Pozițiile elementelor impare. [0,1,-3,-2,8,-1,6,1] → [1,2,5,7]

8 L3·10

multDigits

Produsul cifrelor dintr-un string. "The time is 4:25" → 40; fără cifre → 1

Soluții 1–3: funcții și recursivitate

1 · fizzbuzz

```haskell
fizzbuzz :: Integer -> String
fizzbuzz n
  | n `mod` 15 == 0 = "FizzBuzz"
  | n `mod` 3  == 0 = "Fizz"
  | n `mod` 5  == 0 = "Buzz"
  | otherwise       = ""

fizzbuzzIf :: Integer -> String
fizzbuzzIf n =
  if n `mod` 15 == 0 then "FizzBuzz"
  else if n `mod` 3 == 0 then "Fizz"
  else if n `mod` 5 == 0 then "Buzz"
  else ""
```

2 · tribonacci

```haskell
tribonacci :: Integer -> Integer
tribonacci 1 = 1
tribonacci 2 = 1
tribonacci 3 = 2
tribonacci n = tribonacci (n - 1)
             + tribonacci (n - 2)
             + tribonacci (n - 3)
```

3 · binomial

```haskell
binomial :: Integer -> Integer -> Integer
binomial _ 0 = 1
binomial 0 _ = 0
binomial n k = binomial (n - 1) k
             + binomial (n - 1) (k - 1)
```

Soluții 4–5: recursivitate pe liste

4 · nrVocale

```
nrVocale :: [String] -> Int
nrVocale [] = 0
nrVocale (s:ss)
  | s == reverse s = nrVoc s + nrVocale ss
  | otherwise      = nrVocale ss
  where
    nrVoc "" = 0
    nrVoc (c:cs)
      | c `elem` "aeiouAEIOU" = 1 + nrVoc cs
      | otherwise             = nrVoc cs
```

5 · după fiecare par

```
adaugaDupaPar :: Int -> [Int] -> [Int]
adaugaDupaPar _ [] = []
adaugaDupaPar n (x:xs)
  | even x    = x : n : adaugaDupaPar n xs
  | otherwise = x : adaugaDupaPar n xs
```

verificare

```
ghci> nrVocale ["sos","civic","palton","desen","aerisirea"]
9
ghci> adaugaDupaPar 3 [1,2,3,4,5,6]
[1,2,3,3,4,3,5,6,3]
```

Soluții 6–7: descrieri de liste și zip

6 · listadiv

```haskell
divizori :: Int -> [Int]
divizori n = [d | d <- [1..n], n `mod` d == 0]

listadiv :: [Int] -> [[Int]]
listadiv l = [divizori x | x <- l]
```

verificare

```
ghci> listadiv [1,4,6,8]
[[1],[1,2,4],[1,2,3,6],[1,2,4,8]]
ghci> pozitiiImpareComp [0,1,-3,-2,8,-1,6,1]
[1,2,5,7]
```



Truc: `zip [0..] l` atașează fiecărui element poziția lui. Îl vei folosi des.

7 · pozitiiImpare

```haskell
pozitiiImpareRec :: [Int] -> [Int]
pozitiiImpareRec l = aux 0 l
  where
    aux _ [] = []
    aux i (x:xs)
      | odd x     = i : aux (i + 1) xs
      | otherwise = aux (i + 1) xs

pozitiiImpareComp :: [Int] -> [Int]
pozitiiImpareComp l =
  [i | (i, x) <- zip [0..] l, odd x]
```

Soluția 8 și o întrebare din laborator

8 · multDigits

```haskell
import Data.Char (isDigit, digitToInt)

multDigitsRec :: String -> Int
multDigitsRec [] = 1
multDigitsRec (c:cs)
  | isDigit c = digitToInt c * multDigitsRec cs
  | otherwise = multDigitsRec cs

multDigitsComp :: String -> Int
multDigitsComp s = product [digitToInt c | c <- s, isDigit c]
```

verificare

```
ghci> multDigitsComp "The time is 4:25"
40
ghci> multDigitsRec "No digits here!"
1
```



De ce nu merge doar cu o descriere de listă?

O descriere de listă produce mereu o listă. Pentru un număr îți trebuie o funcție de agregare:

```haskell
pozitiveComp :: [Int] -> Int
pozitiveComp l =
  length [x | x <- l, x > 0]
```

Agregarea în general (fold) e tema prezentării 2.

08

Recapitulare

Capcanele frecvente, fițuica prezentării și un quiz de final.



Capcane frecvente

`f -1`

se citește f - 1. Scrie f (-1).

`f x:xs = …`

lipsesc parantezele: f (x:xs)

`fact n înainte de fact 0`

cazul general prinde tot: recursie infinită

`tab-uri amestecate`

parse error ciudat; folosește doar spații

`'a' vs "a"`

'a' e Char; "a" e String, adică [Char]

`Int vs Integer`

Int face overflow fără avertisment

`head [], tail []`

excepție la rulare; tratează cazul []

`fst (1,2,3)`

fst și snd merg doar pe perechi

Fițuica prezentării 1

GHCi

```
:l f.hs    încarcă
:r         reîncarcă
:t e       tipul lui e
:i nume    informații
:m + M     adaugă modul
:q         ieșire
Ctrl+C     oprește

import Data.List
import Data.Char

-- comentariu
{- bloc -}
```

Sintaxă

```
let a = 1 in a + 1

f x = y + 1
  where y = 2 * x

if c then a else b

case x of
  0 -> "zero"
  _ -> "altceva"

f x
  | x > 0     = 1
  | otherwise = 0
```

Tipuri

```
Int      64 de biți
Integer  nelimitat
Double   real
Bool     True, False
Char     'a'
String   "abc"

(a, b)   tuplu
[a]      listă
a -> b   funcție

Num a => a
Ord a => a -> a -> a
fromIntegral
```

Liste

```
[]  x:xs  [1,2,3]
++  !!  elem
head  tail
take  drop
length  reverse

[1..10]  [1,3..]
['a'..'z']

[e | x <- l, p x]
zip [0..] l

f []     = ...
f (x:xs) = ...
```

Quiz de final

1. Ce tip are `[('a', True)]` ?

2. Ce întoarce `[x * 2 | x <- [1..5], odd x]` ?

3. De ce dă eroare `foo x x = x` ?

4. Ce întoarce `take 3 (zip [1..] "haskell")` ?

5. Cât face `3 + 5 * 4 : [6] ++ [9]` ?

Quiz de final: răspunsuri

1	`[(Char, Bool)]`	o listă de perechi
2	`[2,6,10]`	x ia valorile impare 1, 3, 5, care se dublează
3	**șabloanele sunt liniare**	x nu poate apărea de două ori; folosește o gardă
4	`[(1,'h'),(2,'a'),(3,'s')]`	lazy: din [1..] se iau doar trei elemente
5	`[23,6,9]`	5 * 4 = 20, + 3 = 23, apoi 23 : ([6] ++ [9])

URMEAZĂ · PREZENTAREA 2

Funcții de ordin înalt

Lambda, currying, compunere, map, filter și fold
(Curs 3–4, Laborator 4–5)

Până data viitoare

Termină exercițiile din Lab 3, inclusiv cele Extra (permutări, combinări, n dame). Citește din Learn You a Haskell: Starting Out, Types and Typeclasses, Syntax in Functions, Recursion.